Lập trình mạng · Chương 4

# RPC: truyền tham số, idempotency và truy cập đồng thời trong RMI

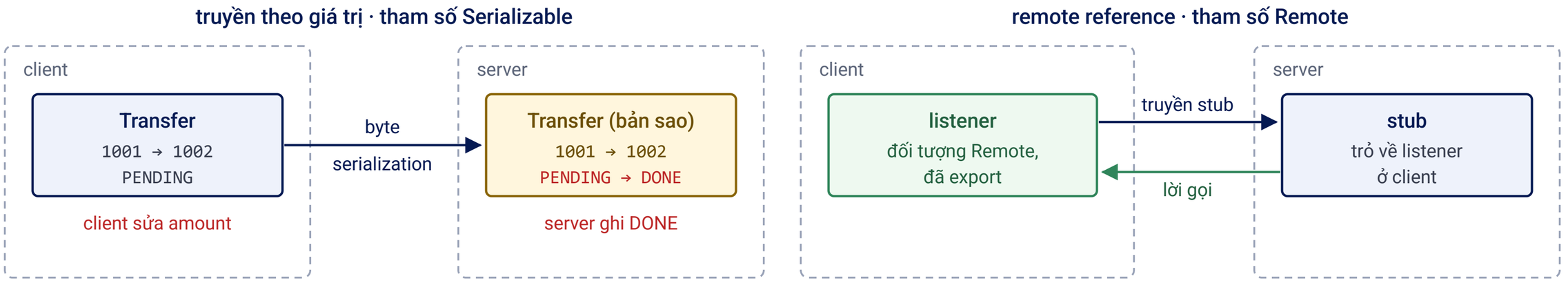
TS. Đặng Ngọc Hùng

# Thêm lệnh chuyển tiền: ba câu hỏi mới cho lời gọi từ xa

1. Tham số `Transfer` đi qua mạng: server nhận **chính đối tượng** của client hay một **bản sao**?
2. Gọi lại sau RemoteException: tài khoản có bị **trừ hai lần**?
3. Nhiều client cùng rút tiền một tài khoản: remote object có còn **đúng** khi các lời gọi chạy song song?

> **Câu hỏi của phần:** RMI truyền tham số thế nào, gọi lại sao cho an toàn, và remote object chịu truy cập đồng thời ra sao?

# Tham số Serializable thành bản sao, tham số Remote thành stub




hai đối tượng độc lập:
sửa ở một đầu không ảnh hưởng đầu kia

lời gọi qua stub ở server
chạy trên đối tượng gốc ở client

**Truyền theo giá trị**: tham số Serializable thành bản sao độc lập. **Remote reference**: tham số Remote thành stub trỏ về đối tượng gốc.

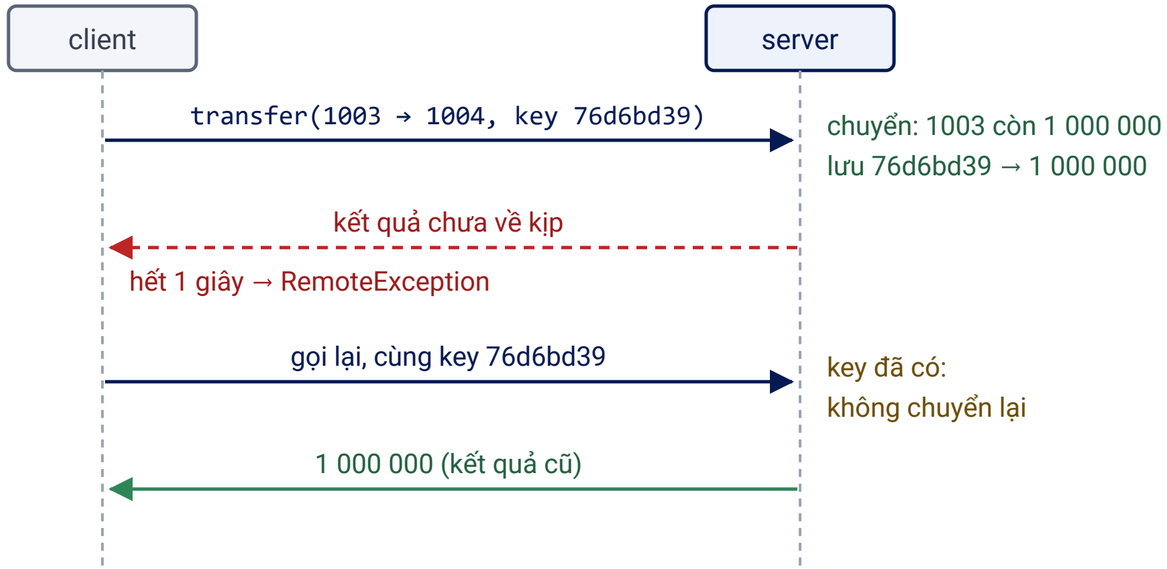
# Idempotency key cho phép gọi lại mà không chuyển tiền hai lần

### gọi hai lần thay vì một lần

`balance("1001")`
số dư không đổi → idempotent

đặt hạn mức = 10 triệu
vẫn là 10 triệu → idempotent

`transfer(1001 → 1002, 1000000)`
bị trừ 2 triệu → không idempotent




**Idempotency key**: một key cho mỗi lệnh, dùng lại khi gọi lại; server lưu key đã xử lý và trả kết quả cũ.

# Sửa bản sao ở một đầu, đầu kia không thấy

Java

```
Transfer transfer = new Transfer("1001", "1002", 1_000_000);
long left = bank.transfer(transfer, null);  // transfer đi qua serialization: server nhận một bản sao
System.out.println("server: balance of 1001 = " + left + "; client transfer: " + transfer);
transfer.amount = 9_999_999;  // chỉ sửa bản của client
System.out.println("client transfer after edit: " + transfer);
System.out.println("server: balance of 1001 = " + bank.balance("1001"));
```

Kết quả chạy

```
server: balance of 1001 = 4000000; client transfer: 1001 -> 1002: 1000000 (PENDING)
client transfer after edit: 1001 -> 1002: 9999999 (PENDING)
server: balance of 1001 = 4000000
```

# Không có key, tài khoản bị trừ hai lần; có key, chỉ một lần

```java
long left = requestId == null
        ? apply(transfer)  // không có idempotency key: mỗi lời gọi là một lần chuyển
        : processed.computeIfAbsent(requestId, id -> apply(transfer));  // key đã xử lý: trả kết quả cũ
if (pause && paused.add(transfer.from)) {
    sleep(1_500);  // mô phỏng server chậm: đã chuyển nhưng chưa trả kết quả
}
```

Kết quả chạy

```
no idempotency key: transfer 1000000 from 1001, balance before 5000000
  attempt 1: UnmarshalException, cause java.net.SocketTimeoutException: Read timed out
  attempt 2: balance 3000000
  balance after 3000000
idempotency key 76d6bd39: transfer 1000000 from 1003, balance before 2000000
  attempt 1: UnmarshalException, cause java.net.SocketTimeoutException: Read timed out
  attempt 2: balance 1000000
  balance after 1000000
```

# RMI có thể chạy các lời gọi đồng thời: remote object phải an toàn

Java

```
long current = unsafeAccounts.get(account);  // đọc rồi ghi: hai thread có thể đọc cùng một số dư
unsafeAccounts.put(account, current - amount);

return accounts.compute(account, (k, current) -> current - amount);  // đọc và ghi nguyên tử cho từng tài khoản
```

Kết quả chạy

```
UnsafeBank: 2000000 - 2000 x 1000 -> balance 41000
Bank: 2000000 - 2000 x 1000 -> balance 0
```

- RMI **không quy định** lời gọi chạy trên thread nào: hai lời gọi đến cùng lúc chạy trên hai thread
- HashMap đọc rồi ghi: mất 41 lần rút; **ConcurrentHashMap**.compute: đọc và ghi nguyên tử cho từng tài khoản

# Câu hỏi 1

**Phương thức RMI ở server đặt trạng thái của tham số Transfer (Serializable) mà nó nhận được thành DONE. Sau lời gọi, Transfer của client ra sao?**

A. Trạng thái thành DONE, vì RMI truyền tham chiếu tới đối tượng của client.

B. Trạng thái vẫn là PENDING, vì server sửa trên bản sao của nó.

C. Trạng thái thành DONE ở lần client gọi balance tiếp theo.

D. Lời gọi ném RemoteException vì tham số bị sửa.

# Câu hỏi 1

## Phương thức RMI ở server đặt trạng thái của tham số Transfer (Serializable) mà nó nhận được thành DONE. Sau lời gọi, Transfer của client ra sao?

A. Trạng thái thành DONE, vì RMI truyền tham chiếu tới đối tượng của client.

B. Trạng thái vẫn là PENDING, vì server sửa trên bản sao của nó.

C. Trạng thái thành DONE ở lần client gọi balance tiếp theo.

D. Lời gọi ném RemoteException vì tham số bị sửa.

Tham số Serializable được truyền theo giá trị: server deserialization thành bản sao riêng. Muốn client biết kết quả thì server trả qua giá trị trả về.

# Câu hỏi 2

**Client gọi phương thức transfer, nhận RemoteException rồi gọi lại. Vì sao lần gọi lại phải mang cùng idempotency key với lần đầu?**

A. Để server xác thực client trước khi chuyển tiền.

B. Để RMI dùng lại kết nối TCP của lần đầu.

C. Vì không có key thì lần gọi lại chắc chắn thất bại.

D. Vì lần đầu có thể đã được thực hiện; cùng key giúp server nhận ra lần gọi lặp và trả kết quả cũ.

# Câu hỏi 2

## Client gọi phương thức transfer, nhận RemoteException rồi gọi lại. Vì sao lần gọi lại phải mang cùng idempotency key với lần đầu?

A. Để server xác thực client trước khi chuyển tiền.

B. Để RMI dùng lại kết nối TCP của lần đầu.

C. Vì không có key thì lần gọi lại chắc chắn thất bại.

**D. Vì lần đầu có thể đã được thực hiện; cùng key giúp server nhận ra lần gọi lặp và trả kết quả cũ.**

RemoteException không cho biết server đã thực hiện hay chưa. Lần gọi lại mang cùng key để server nhận ra lệnh đã thực hiện và không chuyển tiền lần nữa; key mới sẽ bị coi là lệnh mới.

# Tổng kết

1. Tham số Serializable: **truyền theo giá trị**, hai bản độc lập; tham số Remote: **remote reference**, server nhận stub

2. Chuyển tiền không có **idempotency**: gọi lại cần **idempotency key**; server lưu key và trả kết quả cũ

3. RMI có thể chạy các lời gọi đồng thời: remote object dùng **ConcurrentHashMap**, đọc và ghi nguyên tử

**Từ khóa cần nhớ** truyền theo giá trị · remote reference · idempotency · idempotency key · ConcurrentHashMap

# Mở rộng

### Chủ đề nâng cao

- Distributed garbage collection của RMI
- Callback từ server về client bằng remote reference

### Bài tập mở rộng

Viết test gửi cùng một lệnh chuyển tiền 100 lần đồng thời từ 10 client với cùng idempotency key; kiểm tra tài khoản chỉ bị trừ một lần.

Tiếp theo

# Web service: SOAP, hợp đồng WSDL và message XML